

TABLE: ACTIVITY

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
1	3	2017-06-25	1
3	1	2016-03-02	0

QUESTION

Write an SQL query to report for each player and date, how many games played so far by the player. That is, the total number of games played by the player until that date.

Rules:

- Return the result table in any order.

EXPECTED OUTPUT

player_id	event_date	games_played_so_far
1	2016-03-01	5
1	2016-05-02	11
1	2017-06-25	12
3	2016-03-02	0

Think: `SUM(games_played) OVER (PARTITION BY player_id ORDER BY event_date)`.

Write SQL Solution Here ↓

1 Select player_id, event_date,
2 Sum(games_played) over (partition by player_id
3 order by event_date) as games_played_so_far
4 from Activity

6

7 Self join

9 Select a1.player_id,
10 a1.event_date,
11 Sum(a2.games_played) as games_played_so_far
12 from Activity a1
13 join activity a2
14 on a1.player_id = a2.player_id
15 And a2.event_date <= a1.event_date.
16 group By a1.player_id, a1.event_date;

18

19

20

21

22

TABLE: WEATHER

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

QUESTION

Write an SQL query to find all dates' Id with higher temperatures compared to its previous dates (yesterday).

Rules:

- Return the result table in any order.

EXPECTED OUTPUT

id
2
4

Think: LAG() function or Self-JOIN with DATEADD/DATEDIFF.

1 Self join .

2
3 Select w1.id
4 from Weather w1
5 join weather w2

6 on w1.recordDate = DATE_ADD(w2.recordDate, Interval 1 day)
7 where w1.temperature > w2.temperature;

8
9 we can also use

10
11 on DATEDIFF(w1.recordDate, w2.recordDate) = 1 ✓

12
13 LAG() (window function) .

14
15 Select recordDate
16 from (
17 select recordDate,
18 temperature,
19 LAG(temperature) OVER (ORDER BY recordDate)
20 from weather as Prev_tem
21) t
22 where temperature > prev_tem;

23. Game Play Analysis IV

TABLE: ACTIVITY

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-03-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

QUESTION

Write an SQL query to report the fraction of players that logged in again on the day after the day they first logged in, rounded to 2 decimal places.

Rules:

- You need to count the number of players that logged in for at least two consecutive days starting from their first login date, then divide that number by the total number of players.

EXPECTED OUTPUT

fraction
0.33

Think: LEAD() to find next date, or MIN(event_date) to find first login.

select
round(sum(player_login)/count(distinct player_id), 2) as fraction

from (select player_id,
datediff(event_date, min(event_date) over (partition 4
by player_id)) = 1 as player_login,
from activity) as new_table

Short but Tricky — Most Asked SQL Pattern

Write SQL Solution Here ↓

```

1
2 select
3     round(count(distinct Activity.player_id) /
4           (select count(distinct player_id from Activity)
5            as fraction
6            from Activity join (
7                                select player_id,
8                                   min(event_date) as first_date
9                                from Activity
10                               group by player_id) f
11           on Activity.player_id = f.player_id
12           AND Activity.event_date = DATE_ADD(
13              f.first_date, Interval 1 Day)
14
15 ① return first day login (min())
16 ② original activity table (join) each player's first login
17    with the condition
18    Activity.player_id = f.player_id
19 ③ . and condition is activity date should exactly
20    one day after login.
21
22

```

⑥

Total no of unique player also divide
to get the fraction
round off to 2 decimals.